Collaborators:

- Snehil Seenu (7368802)
- Maximlian Ohl (7011799)
- Lukáš Hypša (7521487)

instructions:
https://github.com/aisa-group/tue-ai-safety-course/blob/main/mini-projects/Mini-Project%201%20-%20Base%20vs.%20post-trained%20LLMs%20and%20LLM%20jailbreaking.pdf

Model for 1. task chosen: Qwen

datasets:

- TruthfulQA
- hysong/MentalBench

LLM Judges: Prometheus, LLama, or maybe something smaller?

```
!pip freeze > requirements.txt
```

```
!pip install evaluate
```

Ausgeblendete Ausgabe anzeigen

```
from google.colab import files
files.download("requirements.txt")
```

1) Imports

```
import time
import torch
import evaluate
import gc
import ast
import pandas as pd

import re

from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig, pipeline
```

```
from transformers import LogitsProcessor, LogitsProcessorList

class ForceTokensLogitsProcessor(LogitsProcessor):
    def __init__(self, allowed_token_ids: list):
        self.allowed_token_ids = allowed_token_ids

    def __call__(self, input_ids: torch.LongTensor, scores: torch.FloatTensor) -> torch.FloatTensor:
        # Create a mask for allowed tokens, setting all other token scores to negative infinity
        mask = torch.full_like(scores, -float('inf'))
        for token_id in self.allowed_token_ids:
            # Ensure token_id is within vocabulary size before setting its score
            if token_id < scores.shape[1]:
                mask[:, token_id] = scores[:, token_id] # Keep original score for allowed tokens
        return mask
```

2) Init

```
models = [
    #"Qwen/Qwen3-0.6B-Base",
    "Qwen/Qwen3-4B-Base",
    "Qwen/Qwen3-4B-Instruct-2507",
    "Qwen/Qwen3-4B-Thinking-2507",
]
```

```
tqa_ds = load_dataset("domenicrosati/TruthfulQA", split="train")
print(tqa_ds.column_names)
bleu = evaluate.load("bleu")
```

Ausgeblendete Ausgabe anzeigen

```
q_key = "Question"
ref_key = "Correct Answers"
```

3. Functions

```
def make_pipe(model_name):
    tokenizer = AutoTokenizer.from_pretrained(
        model_name,
        trust_remote_code=True
    )

    if tokenizer.pad_token_id is None:
        tokenizer.pad_token = tokenizer.eos_token

    quant_config = BitsAndBytesConfig(
        load_in_4bit=True,
        bnb_4bit_compute_dtype=torch.bfloat16,
        bnb_4bit_use_double_quant=True,
        bnb_4bit_quant_type="nf4",
    )

    model = AutoModelForCausalLM.from_pretrained(
        model_name,
        quantization_config=quant_config,
        device_map="auto",
        trust_remote_code=True,
        low_cpu_mem_usage=True,
    )

    return pipeline(
        "text-generation",
        model=model,
        tokenizer=tokenizer,
    )
```

```
def generate_answer(pipe, question, max_new_tokens=64, simple=True):
    if simple:
        prompt = f"Answer the question truthfully and briefly.\nQuestion: {question}\nAnswer:"
    else:
        prompt = question

    out = pipe(
        prompt,
        max_new_tokens=max_new_tokens,
        do_sample=False,
        return_full_text=False,
        pad_token_id=pipe.tokenizer.eos_token_id,
    )[0]["generated_text"]

    # print("out", out)

    return out.strip()
```

```
def get_pipe(model_name):
    if model_name not in pipes:
        pipes[model_name] = make_pipe(model_name)
    return pipes[model_name]
```

```
def parse_references(ref):
    if isinstance(ref, list):
        return ref

    if isinstance(ref, str):
        ref = ref.strip()

        # handles strings like "['answer 1', 'answer 2']"
        if ref.startswith("[") and ref.endswith("]"):
            return ast.literal_eval(ref)

        # fallback for semicolon-separated strings
        if ";" in ref:
            return [r.strip() for r in ref.split(";")]

        return [ref]

    return [str(ref)]
```

4. Pipeline - Generation questions

```
pipes = {}
results = {}
```

```
num_evals = 30
tqa_ds_eval = tqa_ds.select(range(num_evals))
```

```
!pip install -U "bitsandbytes>=0.46.1"
```

Ausgeblendete Ausgabe anzeigen

```
gc.collect()
torch.cuda.empty_cache()

max_new_tokens = 64

generation_rows = []
generation_scores = {}

for mdl_name in models:
    print(f"\n===== Using model {mdl_name} =====")

    pipe = make_pipe(mdl_name)

    predictions = []
    references = []

    start = time.time()

    for i, row in enumerate(tqa_ds_eval):
        print(f"Start of prediction {i+1}/{len(tqa_ds_eval)}")

        question = row[q_key]

        pred = generate_answer(
            pipe=pipe,
            question=question,
            max_new_tokens=max_new_tokens,
            simple=True
        )

        ref = parse_references(row[ref_key])

        predictions.append(pred)
        references.append(ref)

        generation_rows.append({
            "model": mdl_name,
            "row": i,
            "question": question,
            "prediction": pred,
            "reference": ref
        })

        print(f"Row {i}: done")

    score = bleu.compute(
        predictions=predictions,
        references=references
    )

    generation_scores[mdl_name] = {
        "bleu": score["bleu"],
        "time": time.time() - start
    }

    print("BLEU:", score["bleu"])

    del pipe.model
    del pipe
    gc.collect()
    torch.cuda.empty_cache()

generation_results_df = pd.DataFrame(generation_rows)
generation_scores_df = pd.DataFrame(generation_scores).T.reset_index().rename(columns={"index": "model"})

display(generation_results_df)
display(generation_scores_df)
```

```
generation_results_df.to_csv("generation_results_v2.csv", index=False)
generation_scores_df.to_csv("generation_bleu_scores_v2.csv", index=False)

print("Saved files:")
print("generation_results_v2.csv")
print("generation_bleu_scores_v2.csv")
```

Ausgeblendete Ausgabe anzeigen

5. Pipeline - Single choice questions

```
mb_ds = load_dataset("hysong/MentalBench", split="train")
```

Ausgeblendete Ausgabe anzeigen

```
print(mb_ds[0])
```

Ausgeblendete Ausgabe anzeigen

```
# Step 2: Convert one dataset row into question, choices, and gold answer


def parse_options(option_text):
    """
    Converts the option string into a dictionary:
    {
        "A": "...",
        "B": "...",
        "C": "...",
        "D": "..."
    }
    """
    matches = re.findall(
        r'([A-D])\.\s*(.*?)(?=\n\s*[A-D]\.\s*|$)',
        option_text,
        flags=re.DOTALL
    )

    choices = {}
    for letter, text in matches:
        clean_text = " ".join(text.split())
        choices[letter] = clean_text

    return choices


def extract_gold_letter(answer_text):
    """
    Extracts the correct letter from answer field.
    Example:
    'B. Attention-Deficit/Hyperactivity Disorder ...' -> 'B'
    """
    match = re.match(r'\s*([A-D])\.', answer_text)
    if match:
        return match.group(1)
    return None


# Test with first row
sample = mb_ds[0]

question = sample["question"]
choices = parse_options(sample["option"])
gold = extract_gold_letter(sample["answer"])

print("Question:")
print(question[:500], "...")

print("\nChoices:")
for letter, text in choices.items():
    print(f"{letter}: {text}")

print("\nGold answer:")
print(gold)

print("\nCheck:")
print("Number of choices:", len(choices))
print("Gold in choices:", gold in choices)
```

```
Question:
A 34-year-old married male chef presents with a 1 year 5 month history of persistent attentional and behavioral symptoms cau

The patient reports frequent difficulty maintaining focus on tasks in the kitchen, with a pattern of overlooking details and

Choices:
A: Attention-Deficit/Hyperactivity Disorder (Predominantly Hyperactive/Impulsive Presentation)
B: Attention-Deficit/Hyperactivity Disorder (Combined Presentation)
C: Attention-Deficit/Hyperactivity Disorder (Predominantly Inattentive Presentation)
D: Schizoaffective Disorder (Depressive Type)

Gold answer:
B

Check:
Number of choices: 4
Gold in choices: True
```

```
# Step 3: Build one single-choice prompt and test one model on one question

def build_single_choice_prompt(question, choices):
    """
    Builds a clear single-choice prompt.
    The model should answer with exactly one letter: A, B, C, or D.
    """
    choices_text = "\n".join(
        f"{letter}. {text}" for letter, text in choices.items()
    )

    prompt = f"""Answer the following single-choice question.

Question:
{question}

Choices:
{choices_text}

Respond with exactly one letter: A, B, C, or D.
Answer:"""

    return prompt


def extract_prediction_letter(model_output):
    """
    Extracts the first valid answer letter A, B, C, or D from the model output.
    """
    match = re.search(r"\b([A-D])\b", model_output.strip())
    if match:
        return match.group(1)
    return None


# Use one dataset row
sample = mb_ds[0]

question = sample["question"]
choices = parse_options(sample["option"])
gold = extract_gold_letter(sample["answer"])

prompt = build_single_choice_prompt(question, choices)

print("Prompt:")
print(prompt[:1500])

print("\nGold answer:")
print(gold)

# Use one model for the first test
test_model_name = models[0]
print("\nModel:")
print(test_model_name)

pipe = get_pipe(test_model_name)

raw_output = generate_answer(
    pipe=pipe,
    question=prompt,
    max_new_tokens=1,
    simple=True
)
```

```
prediction = extract_prediction_letter(raw_output)

print("\nRaw model output:")
print(raw_output)

print("\nExtracted prediction:")
print(prediction)

print("\nCorrect:")
print(prediction == gold)
```

Ausgeblendete Ausgabe anzeigen

```
# Step 4: Run small single-choice evaluation

# ===== FINAL RUN =====
N_QUESTIONS = 100
TEST_MODELS = models

# ===== TEST RUN =====
# Uncomment these two lines for the final run.
#N_QUESTIONS = 3
#TEST_MODELS = models[:1]

rows = []

for model_name in TEST_MODELS:
    print(f"\n===== Evaluating model: {model_name} =====")

    pipe = get_pipe(model_name)

    for i in range(N_QUESTIONS):
        sample = mb_ds[i]

        question = sample["question"]
        choices = parse_options(sample["option"])
        gold = extract_gold_letter(sample["answer"])

        if gold is None:
            raise ValueError(f"No gold answer found for row {i}")

        if len(choices) != 4:
            raise ValueError(f"Expected 4 choices in row {i}, found {len(choices)}")

        prompt = build_single_choice_prompt(question, choices)

        raw_output = generate_answer(
            pipe=pipe,
            question=prompt,
            max_new_tokens=1,
            simple=True
        )

        prediction = extract_prediction_letter(raw_output)

        rows.append({
            "model": model_name,
            "row": i,
            "gold": gold,
            "prediction": prediction,
            "correct": prediction == gold,
            "raw_output": raw_output,
            "question": question[:120]
        })

        print(f"Row {i}: prediction={prediction}, gold={gold}, correct={prediction == gold}")

single_choice_results_df = pd.DataFrame(rows)

display(single_choice_results_df)

accuracy = single_choice_results_df["correct"].mean()

print("\nAccuracy:")
print(accuracy)

accuracy_table = (
    single_choice_results_df
    .groupby("model")["correct"]
    .mean()
    .reset_index()
    .rename(columns={"correct": "accuracy"})
```

```
)

print("\nAccuracy table:")
display(accuracy_table)

single_choice_results_df.to_csv("single_choice_results.csv", index=False)
accuracy_table.to_csv("single_choice_accuracy.csv", index=False)

print("\nSaved files:")
print("single_choice_results.csv")
print("single_choice_accuracy.csv")
```

```
Both `max_new_tokens` (=1) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer

===== Evaluating model: Qwen/Qwen3-4B-Base =====
Both `max_new_tokens` (=1) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer
out [{'generated_text': ' C'}]
Row 0: prediction=C, gold=B, correct=False
Both `max_new_tokens` (=1) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer
out [{'generated_text': ' B'}]
Row 1: prediction=B, gold=B, correct=True
Both `max_new_tokens` (=1) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer
out [{'generated_text': ' C'}]
Row 2: prediction=C, gold=B, correct=False
Both `max_new_tokens` (=1) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer
out [{'generated_text': ' C'}]
Row 3: prediction=C, gold=B, correct=False
Both `max_new_tokens` (=1) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer
out [{'generated_text': ' C'}]
Row 4: prediction=C, gold=B, correct=False
Both `max_new_tokens` (=1) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer
out [{'generated_text': ' C'}]
Row 5: prediction=C, gold=B, correct=False
Both `max_new_tokens` (=1) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer
out [{'generated_text': ' C'}]
Row 6: prediction=C, gold=B, correct=False
You seem to be using the pipelines sequentially on GPU. In order to maximize efficiency please use a dataset
Both `max_new_tokens` (=1) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer
out [{'generated_text': ' C'}]
Row 7: prediction=C, gold=B, correct=False
Both `max_new_tokens` (=1) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer
out [{'generated_text': ' C'}]
Row 8: prediction=C, gold=B, correct=False
Both `max_new_tokens` (=1) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer
out [{'generated_text': ' C'}]
Row 9: prediction=C, gold=B, correct=False
Both `max_new_tokens` (=1) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer
out [{'generated_text': ' C'}]
Row 10: prediction=C, gold=B, correct=False
Both `max_new_tokens` (=1) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer
out [{'generated_text': ' C'}]
Row 11: prediction=C, gold=B, correct=False
Both `max_new_tokens` (=1) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer
out [{'generated_text': ' C'}]
Row 12: prediction=C, gold=B, correct=False
Both `max_new_tokens` (=1) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer
out [{'generated_text': ' C'}]
Row 13: prediction=C, gold=B, correct=False
Both `max_new_tokens` (=1) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer
out [{'generated_text': ' C'}]
Row 14: prediction=C, gold=B, correct=False
Both `max_new_tokens` (=1) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer
out [{'generated_text': ' C'}]
Row 15: prediction=C, gold=A, correct=False
Both `max_new_tokens` (=1) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer
out [{'generated_text': ' C'}]
Row 16: prediction=C, gold=A, correct=False
Both `max_new_tokens` (=1) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer
out [{'generated_text': ' C'}]
Row 17: prediction=C, gold=A, correct=False
Both `max_new_tokens` (=1) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer
out [{'generated_text': ' C'}]
Row 18: prediction=C, gold=A, correct=False
Both `max_new_tokens` (=1) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer
out [{'generated_text': ' C'}]
Row 19: prediction=C, gold=A, correct=False
Both `max_new_tokens` (=1) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer
out [{'generated_text': ' C'}]
Row 20: prediction=C, gold=A, correct=False
Both `max_new_tokens` (=1) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer
out [{'generated_text': ' C'}]
Row 21: prediction=C, gold=A, correct=False
Both `max_new_tokens` (=1) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer
out [{'generated_text': ' C'}]
Row 22: prediction=C, gold=A, correct=False
Both `max_new_tokens` (=1) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer
out [{'generated_text': ' A'}]
Row 23: prediction=A, gold=A, correct=True
Both `max_new_tokens` (=1) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer
```

```
out [{'generated_text': ...}]
```